

Exercise 4 - Theoretical Questions

Question 1: Lazy Lists

IDO KOZIH
IDO UZIYAHU

June 2026

Question 1.1: Lazy Lists

(a) Equivalence Criterion

Two lazy lists are considered equivalent if, after any finite number of applications of `tail`, either both lists are empty, or both lists have a defined `head` and these two `head` values are equal.

(b) Equivalence of `even-square-1` and `even-square-2`

Let

$$L_1 = \text{even-square-1}$$

and

$$L_2 = \text{even-square-2}.$$

We consider an arbitrary natural number T produced by `integers-from 0`.

In L_1 , the number T is first mapped to T^2 , and then the filter checks whether T^2 is even.

In L_2 , the number T is first filtered according to whether T is even, and only then it is mapped to T^2 .

We split into two cases.

If T is even, then T^2 is also even. Therefore, in L_1 , the value T^2 passes the filter. In L_2 , the value T passes the filter and is then mapped to T^2 . Thus, in both lists, the same value T^2 appears.

If T is odd, then T^2 is also odd. Therefore, in L_1 , the value T^2 does not pass the filter. In L_2 , the value T does not pass the filter before the mapping. Thus, in both lists, T contributes no value to the final lazy list.

Hence, for every natural number T , either both lists include the same value T^2 , or both lists exclude it. Since both `map` and `filter` preserve the original order of the remaining elements, both lazy lists produce the same sequence:

$$0, 4, 16, 36, 64, \dots$$

Therefore, according to the equivalence criterion defined in part (a), L_1 and L_2 are equivalent lazy lists.

Question 2: CPS

(a)

A procedure **func** of type $[A_1 * \dots * A_n \rightarrow T \mid 'fail]$ and its Success-Fail-Continuations version **func**\$ of type $[A_1 * \dots * A_n * [T \rightarrow T1] * [Empty \rightarrow T2] \rightarrow T1 \mid T2]$ are equivalent if and only if for every valid sequence $x_1, \dots, x_n$ and any continuation functions **success** and **fail**

1. $(\mathbf{func} \ x_1 \dots x_n)$ will terminate and return value $\text{res} \neq 'fail \iff (\mathbf{func}\$ \ x_1 \dots x_n \text{ success fail})$ results in the evaluation of (success res).
2. $(\mathbf{func} \ x_1 \dots x_n)$ will terminate and return $'fail \iff (\mathbf{func}\$ \ x_1 \dots x_n \text{ success fail})$ results in the evaluation of (fail).
3. $(\mathbf{func} \ x_1 \dots x_n)$ does not terminate $\iff (\mathbf{func}\$ \ x_1 \dots x_n \text{ success fail})$ does not terminate.

2d

We will prove **get-value** and **get-value**\$ equivalence using induction on $n = \text{len}(\text{assoc-list})$.

Base Case

On $n = 0$, (empty? assoc-list) evaluates to **true**. Therefore, **get-value** will terminate and return $'fail$, and **get-value**\$ will evaluate (fail).

Proposition

For every $n > 0$, we will assume that for every **assoc-list** with $\text{len}(\text{assoc-list}) = n$, **get-value** and **get-value**\$ are equivalent.

Induction Step

We will prove that (get-value assoc-list key) and (get-value\$ assoc-list key success fail) are equivalent for an **assoc-list** with $\text{len}(\text{assoc-list}) = n + 1$.

For easier reading, we will refer to (get-value assoc-list key) as **get-value** and (get-value\$ assoc-list key success fail) as **get-value**\$.

Since $\text{len}(\text{assoc-list}) > 0$, **get-value** will evaluate to:

```
(if (eq? key (car (car assoc-list)))
    (cdr (car assoc-list))
    (get-value (cdr assoc-list) key))
```

and `get-value$` will evaluate to:

```
(if (eq? key (car (car assoc-list)))  
    (success (cdr (car assoc-list)))  
    (get-value$ (cdr assoc-list) key success fail))
```

Case 1: `(eq? key (car (car assoc-list)))` evaluates to `#t`

In this case, `get-value` will evaluate to `(cdr (car assoc-list))`, and `get-value$` will evaluate to `(success (cdr (car assoc-list)))`. Therefore, they are equivalent.

Case 2: `(eq? key (car (car assoc-list)))` evaluates to `#f`

Here, `get-value` will return the evaluation of `(get-value (cdr assoc-list) key)`, and `get-value$` will evaluate `(get-value$ (cdr assoc-list) key success fail)`.

By assuming the induction hypothesis for `(cdr assoc-list)`, which has length n , it follows that `(get-value (cdr assoc-list) key)` and `(get-value$ (cdr assoc-list) key success fail)` are equivalent. Therefore, `get-value` and `get-value$` are equivalent.

Question 3 - Logic Programming

3.1 Unification

1.

$$\text{unify}[t(s(s), G, s, p, t(K), s), t(s(G), G, s, p, t(K), U)]$$

We start with the equation:

$$t(s(s), G, s, p, t(K), s) = t(s(G), G, s, p, t(K), U)$$

Initial substitution:

$$S = \{\}$$

Both sides have the same outer operator t and the same number of arguments, so we decompose:

$$s(s) = s(G)$$

$$G = G$$

$$s = s$$

$$p = p$$

$$t(K) = t(K)$$

$$s = U$$

Choose:

$$s(s) = s(G)$$

Both sides have the same outer operator s , so we decompose:

$$s = G$$

This is a variable equation, so we add:

$$G = s$$

and get:

$$S = \{G = s\}$$

Now consider:

$$G = G$$

Applying the current substitution gives:

$$s = s$$

which is trivial and is removed.

The equations

$$s = s \quad p = p$$

are also trivial and are removed.

Now consider:

$$t(K) = t(K)$$

Both sides have the same outer operator t , so we decompose:

$$K = K$$

This is trivial and is removed.

It remains to solve:

$$s = U$$

This is a variable equation, so we add:

$$U = s$$

Composing with the previous substitution gives:

$$S = \{G = s, U = s\}$$

Therefore, the unification succeeds, and the MGU is:

$$\boxed{\{G = s, U = s\}}$$

2.

$$\text{unify}[g(l, M, g, G, U, g, v(M)), g(l, v(U), g, v(M), v(G), g, v(M))]$$

We start with the equation:

$$g(l, M, g, G, U, g, v(M)) = g(l, v(U), g, v(M), v(G), g, v(M))$$

Initial substitution:

$$S = \{\}$$

Both sides have the same outer operator g and the same number of arguments, so we decompose:

$$\begin{aligned}
l &= l \\
M &= v(U) \\
g &= g \\
G &= v(M) \\
U &= v(G) \\
g &= g \\
v(M) &= v(M)
\end{aligned}$$

The equations

$$l = l \quad g = g \quad g = g$$

are trivial and are removed.

Now choose:

$$M = v(U)$$

This is a variable equation. Since M does not occur in $v(U)$, there is no occurs-check failure. Therefore, we add:

$$M = v(U)$$

and get:

$$S = \{M = v(U)\}$$

Now choose:

$$G = v(M)$$

We first apply the current substitution:

$$G = v(M) \quad \Rightarrow \quad G = v(v(U))$$

This is a variable equation. Since G does not occur in $v(v(U))$, we add:

$$G = v(v(U))$$

Composing with the current substitution gives:

$$S = \{M = v(U), G = v(v(U))\}$$

Now choose:

$$U = v(G)$$

Apply the current substitution:

$$U = v(G) \quad \Rightarrow \quad U = v(v(v(U)))$$

This causes an occurs-check failure, because the variable U occurs inside the term $v(v(v(U)))$. Thus, the unification fails.

Failure: occurs-check on $U = v(v(v(U)))$

Therefore, there is no MGU.

3.

$$\text{unify}[m(M, N), n(M, N)]$$

We start with the equation:

$$m(M, N) = n(M, N)$$

Initial substitution:

$$S = \{\}$$

Choose the equation:

$$m(M, N) = n(M, N)$$

This looks like case C, because both sides are compound expressions. Therefore, we check whether they have the same structure: the same outer operator and the same number of arguments.

The left-hand side has the outer operator:

$$m$$

and the right-hand side has the outer operator:

$$n$$

Although both expressions have two arguments, their outer operators are different:

$$m \neq n$$

Therefore, the equation cannot be decomposed, and the unification fails.

Failure: different outer operators m and n

Therefore, there is no MGU.

4.

$$\text{unify}[p([v \mid [V \mid VV]]), p([[v \mid V] \mid VV])]$$

We start with the equation:

$$p([v \mid [V \mid VV]]) = p([[v \mid V] \mid VV])$$

Initial substitution:

$$S = \{\}$$

Both sides have the same outer operator p and the same number of arguments, so this is case C and we decompose:

$$[v \mid [V \mid VV]] = [[v \mid V] \mid VV]$$

Now we compare the two lists by their head and tail.

The left-hand side is:

$$[v \mid [V \mid VV]]$$

so its head is v , and its tail is:

$$[V \mid VV]$$

The right-hand side is:

$$[[v \mid V] \mid VV]$$

so its head is:

$$[v \mid V]$$

and its tail is VV .

Therefore, decomposing the list equation gives:

$$\begin{aligned} v &= [v \mid V] \\ [V \mid VV] &= VV \end{aligned}$$

Now choose the first equation:

$$v = [v \mid V]$$

This fails, because v is an atom, while $[v \mid V]$ is a compound list. They do not have the same structure, and the atom v cannot be unified with a list.

Therefore, the unification fails.

Failure: atom v cannot be unified with the list $[v \mid V]$

Therefore, there is no MGU.

5.

$$\text{unify}[g([T]), g(T)]$$

We start with the equation:

$$g([T]) = g(T)$$

Initial substitution:

$$S = \{\}$$

Both sides have the same outer operator g and the same number of arguments, so this is case C and we decompose:

$$[T] = T$$

This is a variable equation, because one side is the variable T . Equivalently, we would like to add the substitution:

$$T = [T]$$

However, this causes an occurs-check failure, because the variable T appears inside the term $[T]$.

If we tried to apply this substitution, we would get an infinite structure:

$$T = [T] = [[T]] = [[[T]]] = \dots$$

Therefore, the unification fails.

Failure: occurs-check on $T = [T]$

Therefore, there is no MGU.

3.3 Proof Tree

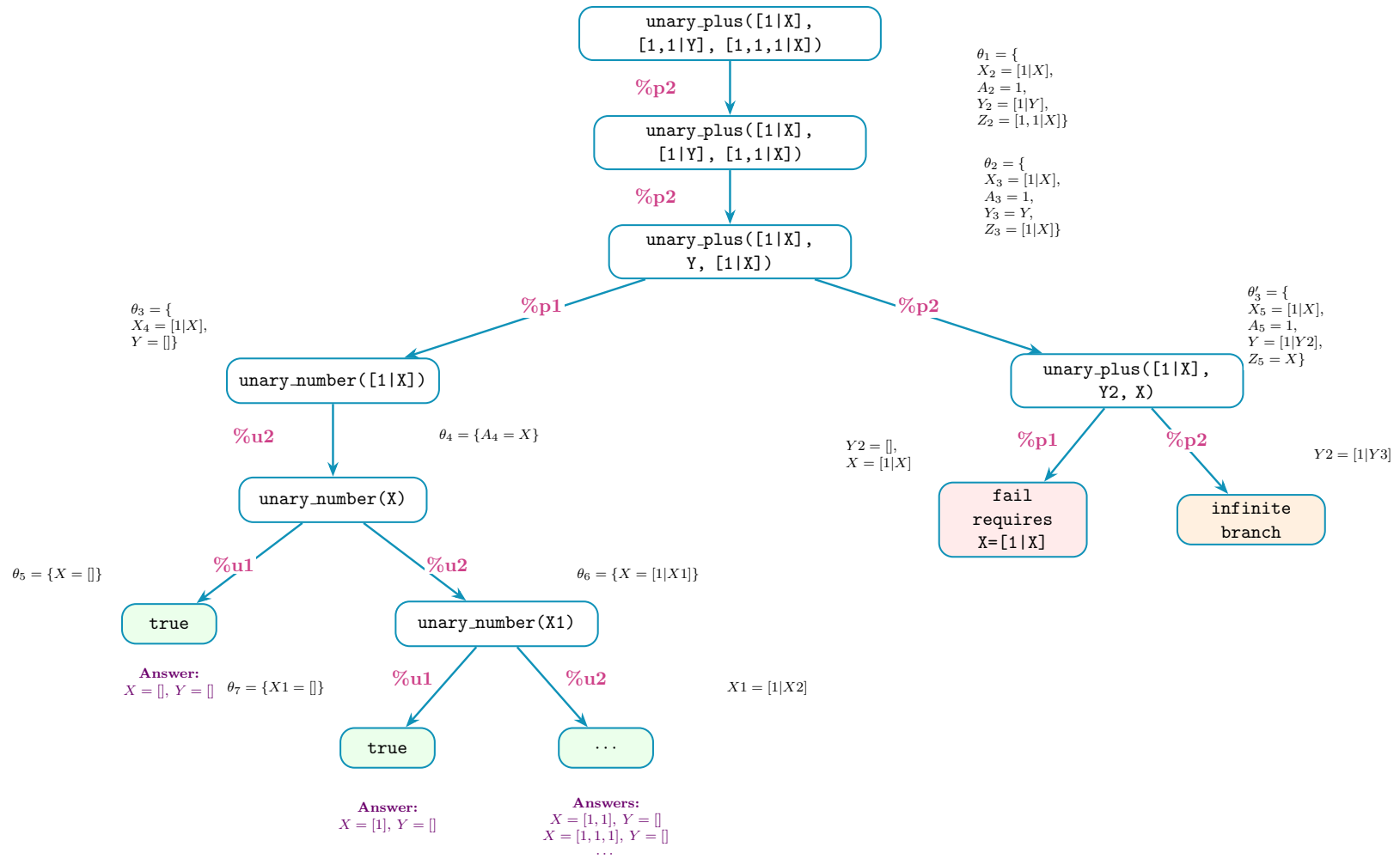
a.

We need to draw the proof tree for the query:

```
?- unary_plus([1|X], [1,1|Y], [1,1,1|X]).
```

The program is:

```
unary_number([]).                               %u1
unary_number([1|A]) :- unary_number(A).         %u2
unary_plus(X, [], X) :- unary_number(X).       %p1
unary_plus(X, [A|Y], [A|Z]) :- unary_plus(X, Y, Z). %p2
```



The success leaves are:

$$\{X = [], Y = []\}, \quad \{X = [1], Y = []\}, \quad \{X = [1, 1], Y = []\}, \quad \dots$$

In general, for every $n \geq 0$:

$$\sigma_n = \{Y = [], X = [1^n]\}$$

where $[1^n]$ denotes a list containing n occurrences of 1.